CSC3002
C/C++ Programming

Pointer and Array

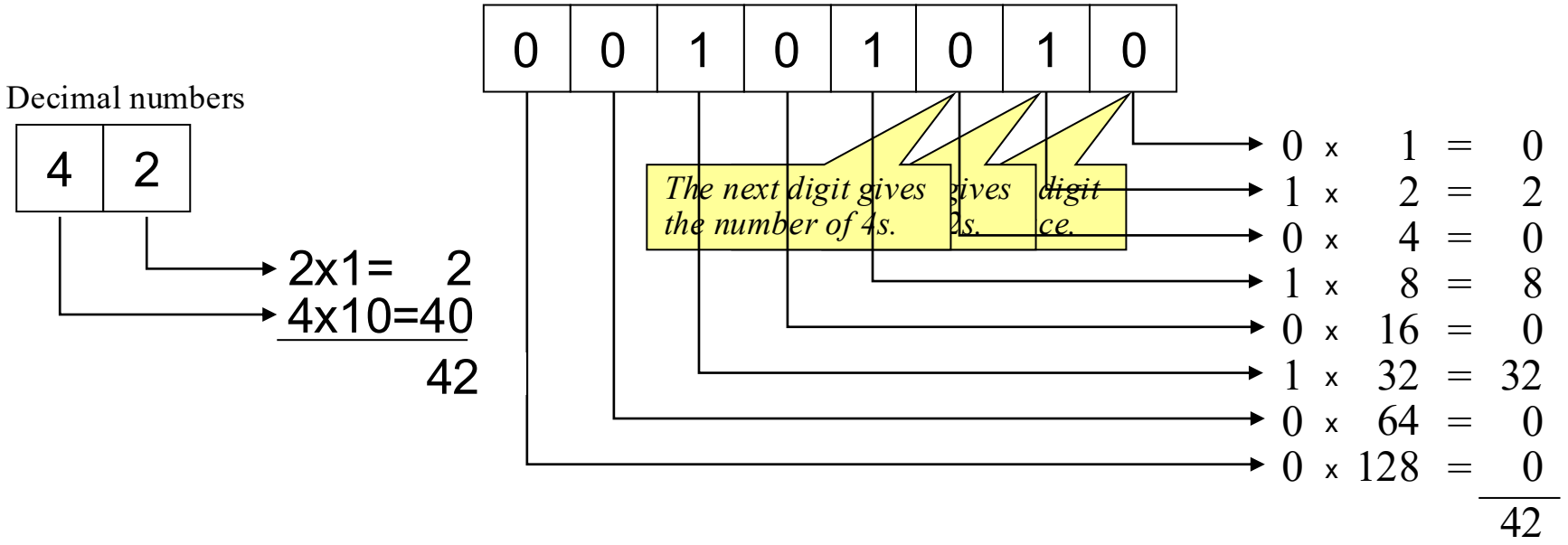
Prof. Dr. Ruizhi Chen， SDS

Office: TxC 717

1. The structure of memory
2. Pointers
3. Arrays
4. Pointer arithmetic

Binary Notation

- Bytes and words can be used to represent integers of different sizes by interpreting the bits as a number in binary notation.
- Binary notation is similar to decimal notation but uses a different base. Decimal numbers use 10 as their base, which means that each digit counts for ten times as much as the digit to its right. Binary notation uses base 2, which means that each position counts for twice as much, as follows:



Numbers and Bases

- The calculation at the end of the preceding slide makes it clear that the binary representation 00101010 is equivalent to the number 42. When it is important to distinguish the base, the text uses a small subscript, like this:

$$00101010_2 = 42_{10}$$

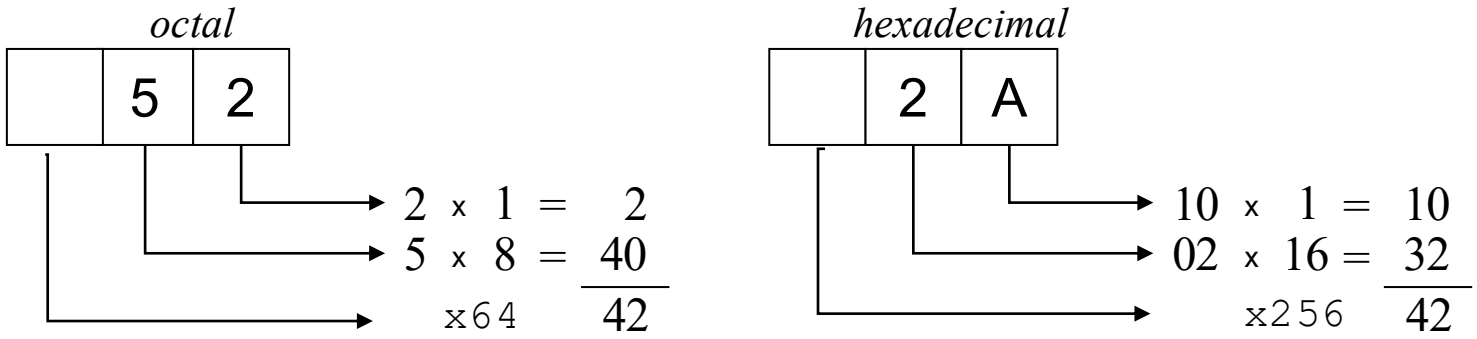
- Although it is useful to be able to convert a number from one base to another, it is important to remember that the number remains the same. What changes is how you write it down.

- The number 42 is what you get if you count how many stars are in the pattern at the right. The number is the same whether you write it in English as forty-two, in decimal as 42, or in binary as 00101010.
- Numbers do not have bases; representations do.



Octal and Hexadecimal Notation

- Because binary notation tends to get rather long, computer scientists often prefer octal (base 8) or hexadecimal (base 16) notation instead. Octal notation uses eight digits: 0 to 7. Hexadecimal notation uses sixteen digits: 0 to 9, followed by the letters A(a) through F(f) to indicate the values 10 to 15.
- The following diagrams show how the number forty-two appears in both octal and hexadecimal notation:



- The advantage of using either octal or hexadecimal notation is that doing so makes it easy to translate the number back to individual bits because you can convert each digit separately.

Numbers and Bases

42

1 - bit 3 - bits 4 - bits

$$00101010_2 = 52_8 = 42_{10} = 2A_{16}$$

2 - bits

$$0222_4 = ?$$

$2x1 = 2$

$2x4 = 8$

$2x16 = 32$

42

Binary

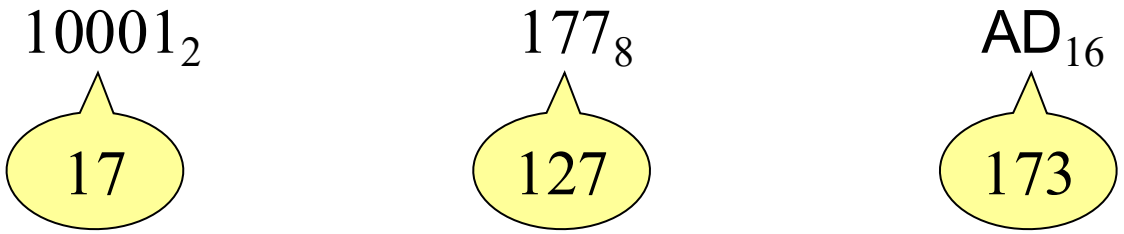
Quaternary

Octal

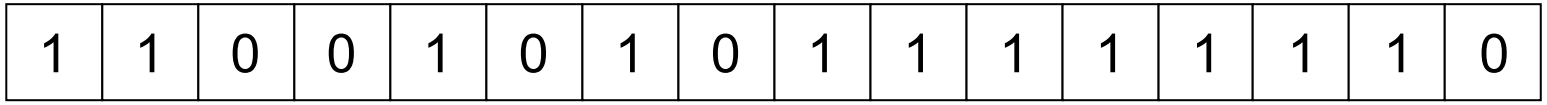
Hexadecimal

Exercises: Number Bases

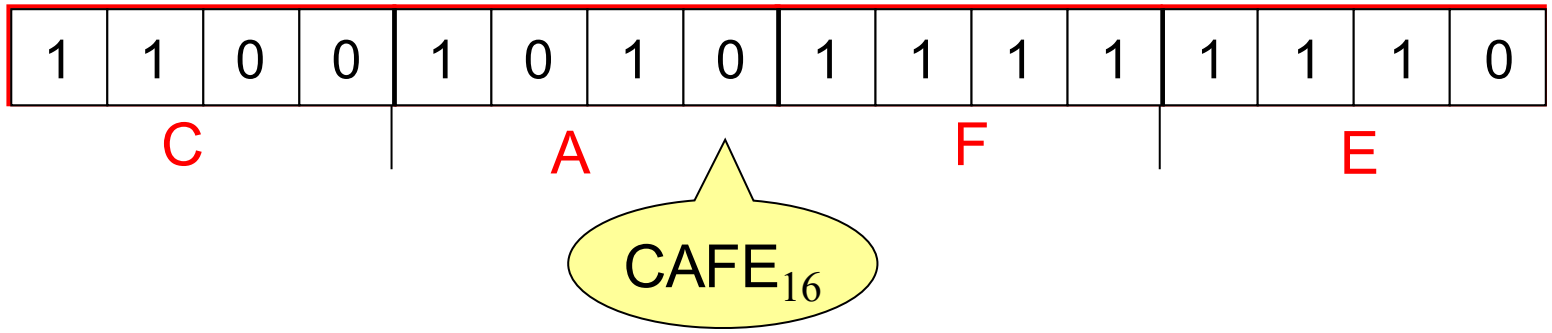
- What is the decimal value for each of the following numbers?



- As part of a code to identify the file type, every Java class file begins with the following sixteen bits:



- How would you express that number in hexadecimal notation?



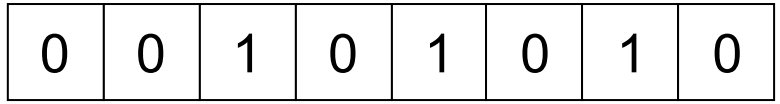
Memory Space: an Analogy

Every modern computer contains some amount of high-speed internal memory that is its principal repository for information. In a typical machine, that memory is built out of special integrated-circuit chips called RAM (Random-Access Memory), which allows the program to use the contents of any memory cell at any time.

Hotel	**Memory**
Bed	Bit
Room	Byte
Floor	Word
Room number	Address
Number of rooms	Memory size

The Structure of Memory

- The fundamental unit of memory inside a computer is called a bit, which is a contraction of the words binary digit. A bit can be in either of two states, usually denoted as 0 and 1.
- The hardware structure of a computer combines individual bits into larger units. In most modern architectures, the smallest addressable unit on which the hardware operates is a sequence of eight consecutive bits called a byte. The following diagram shows a byte containing a combination of 0s and 1s:



- Numbers are stored in still larger units that consist of multiple bytes. The unit that represents the most common integer size on a particular hardware (i.e., the number of bits the CPU can process at one time) is called a (hardware) word. Because machines have different architectures, the number of bits in a word may vary from machine to machine. E.g., a word in x86-64, the 64-bit version of the x86 instruction set, is 64-bit.

Word Size and Address Length

- A word is usually the largest piece of data that can be transferred to/from the memory in a single operation of a particular processor, so the word size is an important characteristic of any specific processor/architecture.
- The largest possible address length, used to designate a location in memory, is typically a word, because this allows one memory address to be efficiently stored in one word.
- Very often, when referring to the word size of a modern computer, one is also describing the address length on that computer. E.g., a computer said to be "32-bit" has a hardware word size of 32 bits, and also usually allows 32-bit memory addresses.
- However, this does not always hold true. Computers can have memory addresses larger or smaller than their word size.

Address Length and Memory Size

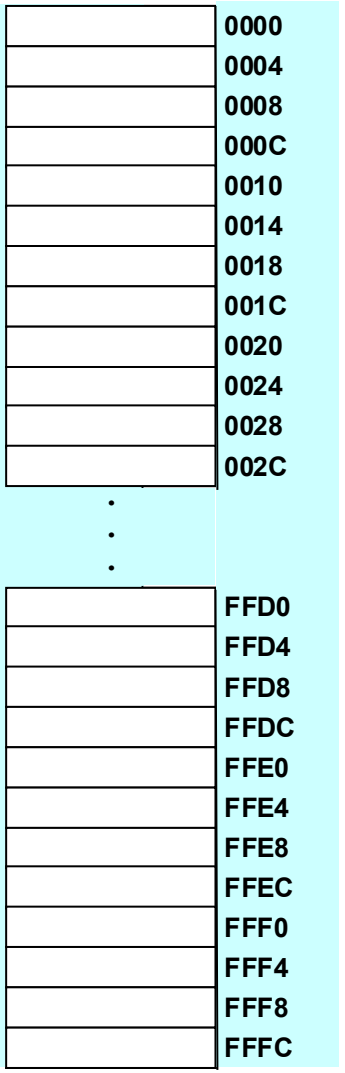
- Although we will make up some four-digit hexadecimal (i.e., 16-bit) numbers as the memory addresses for simplicity in our examples, the actual address lengths may be different on different computers/architectures.

- In theory, modern byte-addressable N-bit computers can address 2^N bytes of memory, but in practice the amount of memory is limited by the CPU, the memory controller, etc.

- Exercise: The theoretical memory sizes in 16-, 32-, and 64-bit machines are?
  - 16 bit → 65, 536 bytes (64 Kilobytes)
  - 32 bit → 4, 294, 967, 295 bytes (4 Gigabytes)
  - 64 bit → 18, 446, 744, 073, 709, 551, 616 (16 Exabytes)

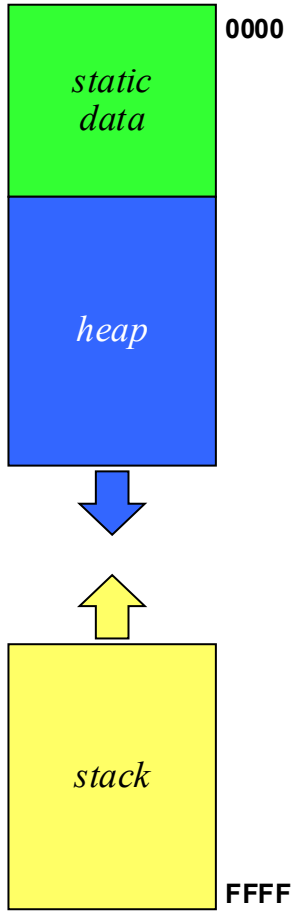
Memory and Addresses

- Every byte inside the primary memory of a machine is identified by a numeric address. The addresses begin at 0 and extend up to the number of bytes in the machine, as shown in the diagram on the right.
- Memory diagrams that show individual bytes are not as useful as those that are organized into words. The revised diagram on the right now includes four bytes (i.e., a 32-bit machine) in each of the memory cells, which means that the address numbers increase by four each time.
- In these slides, addresses are four-digit hexadecimal numbers, which makes them easy to recognize.
- When you create memory diagrams, you don't know the actual memory addresses at which values are stored, but you do know that everything has an address. Just make something up.



The Allocation of Memory to Variables

- When you declare a variable in a program, C++ allocates space for that variable from one of several memory regions.
- One region of memory is reserved for program code and global variables/constants that persist throughout the lifetime of the program. This information is called static data.
- Each time you call a method/function, C++ allocates a new block of memory called a stack frame to hold its local variables. These stack frames come from a region of memory called the stack.
- It is also possible to allocate memory dynamically, as we will describe later. This space comes from a pool of memory called the heap.
- In classical architectures, the stack and heap grow toward each other to maximize the available space.



The Allocation of Memory to Variables

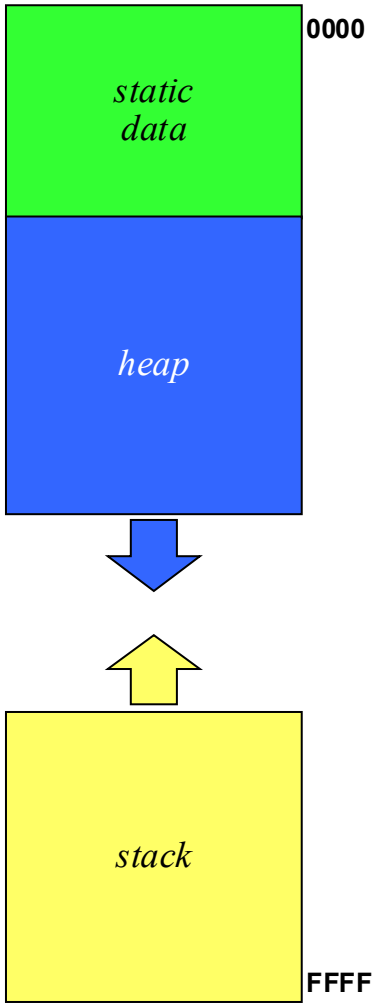
```
#include <iostream>
using namespace std;

int global_int; // Global
void tmpFunction(int function_int);

int main()
{
   global_int = 10;
   tmpFunction(20);
}
void tmpFunction(int function_int)
{                    // stack
   int *p = new int;       // heap
   static int numCalls = 0; // static/global
   numCalls++;
   cout << hex << uppercase;
   cout << "Global var \t\t\t" << &global_int << endl;
   cout << "Static var \t\t\t" << &numCalls << endl;
   cout << "Runtime allocated (Heap) \t" << p << endl;
   cout << "Function (Stack) \t\t" << &function_int << endl;
   delete p;
}
```



```
Global var                      0x102014000
Static var                      0x102014004
Runtime allocated (Heap)        0x1027d5b60
Function (Stack)                0x16ddf108c
```



Data Types in C/C++

The data types that C++ inherits from C:

- Atomic (primitive) types:
  - short, int, long, and their unsigned variants
  - float, double, and long double
  - char
  - bool
- Enumerated types defined using the enum keyword
- Structure types defined using the struct keyword
- Arrays of some base type
- Pointers to a target type

Sizes of the Fundamental Types

- The memory space required to represent a value depends on the type of value. Although the C++ standard actually allows compilers some flexibility, the following sizes are typical:

1 byte (8 bits)	2 bytes (16 bits)	4 bytes (32 bits)	8 bytes (64 bits)	16 bytes (128 bits)
`char` `bool`	`short`	`int` `float`	`long` `double`	`long double`

- Enumerated types are typically assigned the space of an int.
- Structure types have a size equal to the sum of their fields.
- Arrays take up the element size times the number of elements.
- Pointers take up the space needed to hold an address, which is usually the size of a hardware word, e.g., 4 bytes on a 32-bit machine and 8 bytes on a 64-bit machine.
- sizeof(t) returns the actual number of bytes required to store a value of the type t; sizeof x returns the actual memory size of the variable x. sizeof is an operator

Variables



- A variable in C++ is most easily envisioned as a box capable of storing a value. For the following statement:

```
int total = 42;
```

(name is) **total**

(stores at) FFD0 | 42 | (contains an) **int**

- Each variable has the following attributes:
  - A name, which enables you to differentiate one variable from another.
  - A type, which specifies what type of value the variable can contain.
  - A value, which represents the current contents of the variable.
  - For now, let's not worry about the address first.
- The address and type of a named variable are fixed. The value changes whenever you assign a new value to the variable.

Using addresses as data values: lvalue

- In C++, any expression that refers to an internal memory location capable of storing data is called a lvalue, which can appear on the left side of an assignment statement in C++.
- Intuitively, if it cannot be on the left side of an assignment, or if you cannot assign a value to it, it is not a lvalue

```
int x=10, y=20, z=0;
z =x+y;
```

the expression x+y is not a lvalue, by x,y,z are lvalues

- The following properties apply to lvalues in C++:
  - Every lvalue is stored somewhere in memory and therefore has an address.
  - Once it has been declared, the address of an lvalue never changes, even though the contents of those memory locations may change.
  - The address of an lvalue is a value of a pointer variable, which can be stored in memory and manipulated as data.

Value can change, but memory address fixed

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
     int i = 0;
     cout << dec << "i=" << i << "\tmem address: " << hex << uppercase << &i << endl;
     i = 1000;
     cout << dec << "i=" << i << "\tmem address: " << hex << uppercase << &i << endl;

}
```



```
i=0     mem address: 16CFA5094
i=1000  mem address: 16CFA5094
```

Pointers

- In C++, every data item is stored somewhere in memory and can therefore be identified with that address. Because C++ is designed to allow programmers to control data at the lowest level, it makes the fact that memory locations have addresses visible to the programmer.
- A data item whose value is an address in memory is called a pointer, which can be manipulated just like any other kind of data. In particularly, you can assign one pointer value to another, which means that the two pointers end up indicating the same data item.
- Diagrams that involve pointers are typically represented in two different ways:
  - Using memory addresses emphasizes the fact that pointers are just like integers.
  - Access the data value pointed by a pointer using an arrow, like p->x.

Pointers

- In C++, pointers serve several purposes, of which the following are the most important:
  - Pointers allow you to refer to a large data structure in a compact way. Because a memory address typically fits in a few bytes of memory, this strategy offers considerable space savings when the data structures themselves are large. E.g., call by pointers.
  - Pointers make it possible to reserve new memory during program execution. In many applications, it is convenient to acquire new memory as the program runs and to refer to that memory using pointers, which is called dynamic allocation.
  - Pointers can be used to record relationships among data items. Data structures that use pointers to create connections between individual components are called linked structures. Programmers can indicate that one data item follows another in a conceptual sequence by including a pointer to the second item in the internal representation of the first.

Declaring a Pointer Variable

- Pointer variables have a declaration syntax that may at first seem confusing. To declare a variable as a pointer to a particular type as opposed to a variable of that type, all you need to do is add a * in front of the variable name, like this:

```
type*   var;
type *  var;
type   *var;
```

- For example, if you wanted to declare a variable px to be a pointer to a double value, you could do so as follows:

```
double * px;
```

- Similarly, to declare a variable pptr as a pointer to a Point structure, you would write:

```
Point * pptr;
```

Pointer Operators

- C++ includes two built-in operators for working with pointers:
  - The address-of operator (&) is written before a variable name (or any expression to which you could assign a value, an lvalue) and returns the address of that variable.
  - The value-pointed-to operator (*) is written before a pointer expression and returns the actual value of a variable to which the pointer points (dereferencing).
- Suppose, for example, that you have declared and initialized the following variables:

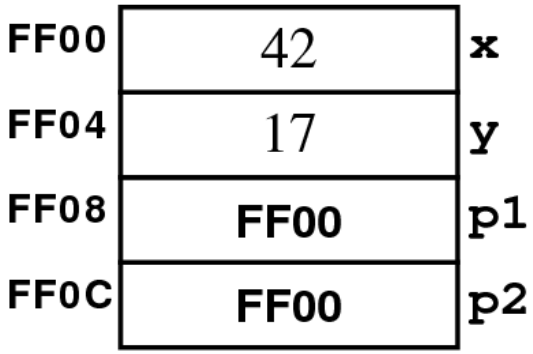
```
double x = 2.5;
double * px = &x;
```

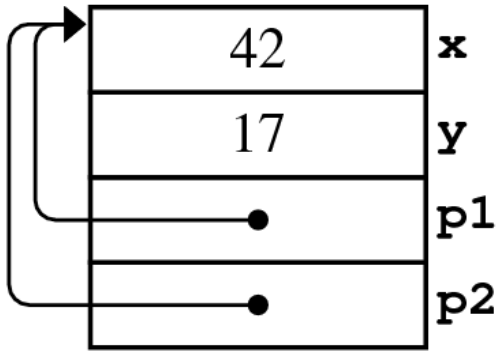
- At this point, the pointer variable px points to the double variable x, and the expression *px is synonymous with the variable x.

```
double y = *px;
```

Pointer Diagrams

```
int x, y;
int *p1, *p2;
x = 42;
y = 163;
p1 = &y;
p2 = &x;
*p1 = 17;
p1 = p2;
*p1 = *p2;
```





Pointers are dangerous

```
#include <iostream>
using namespace std;

int main() {
    int i = 1;
    int * pi = &i;
    double d = 2.1;
    double * pd = &d;
    cout << *pi << endl;
    cout << *pd << endl;
    int * pi1 = (int *) pd;
    cout << *pi1 << endl;
    double * pd1 = (double *) pi;
    cout << *pd1 << endl;
}
```

```
1
2.1

-858993459

3.57212e+230
```

Pointers

- In C++, pointers serve several purposes, of which the following are the most important:
  - Pointers allow you to refer to a large data structure in a compact way. Because a memory address typically fits in a few bytes of memory, this strategy offers considerable space savings when the data structures themselves are large. E.g., call by pointers.
  - Pointers make it possible to reserve new memory during program execution. In many applications, it is convenient to acquire new memory as the program runs and to refer to that memory using pointers, which is called dynamic allocation.
  - Pointers can be used to record relationships among data items. Data structures that use pointers to create connections between individual components are called linked structures. Programmers can indicate that one data item follows another in a conceptual sequence by including a pointer to the second item in the internal representation of the first.

Pointers and Call by Reference

- To swap two integers, the function swap takes its parameters by reference, which means that the stack frame for swap is given the addresses of the calling arguments rather than the values.

```
void swap(int & x, int & y) {
   int tmp = x;
   x = y;
   y = tmp;
}
```

```
swap(n1, n2);
```

- You can simulate the effect of call by reference by making the pointers explicit (call by pointer):

```
void swap(int * px, int * py) {
   int tmp = *px;
   *px = *py;
   *py = tmp;
}
```

```
swap(&n1, &n2);
```

```
void swap2(int *x, int *y)
{
        int *tmp = x;
        x = y;
        y = tmp;
}
```

Not swapped

```
void swap3(int *x, int *y)
{
        int tmp = *x;
        *x = *y;
        *y = tmp;
}
```

Swapped

Swap the values the pointers point to

```
void swap4(int *&x, int *&y)
{
        int *tmp = x;
        x = y;
        y = tmp;
}
```

Not swapped

```
void swap5(int *&x, int *&y)
{
        int tmp = *x;
        *x = *y;
        *y = tmp;
}
```

Swapped

Pointer vs. Reference

	Pointer	Reference
Definition	The memory address of an object	An alternative identifier for an object
Declaration	`int i = 5;` `int * p = &i;`	`int i = 5;` `int & r = i;`
Dereferencing	`*p`	
Has an address in memory	Yes (`&p`)	No (the same as `&i`)
Pointing/referring to nothing	Yes (`NULL`/ `nullptr` since C++11)	No
Reassignments to new objects	Yes	No
Supported by	C and C++	C++

The address-of operator, not a reference.

Pointers to Objects

```
Point pt(3, 4);

Point * pp = &pt;
```

- The above code declares two local variables. The variable pt contains a Point object with the coordinate values 3 and 4. The variable pp contains a pointer to that same Point object.
- To invoke the method, e.g., getX(), of the object:

```
pt.getX();

(*pp).getX();

pp->getX();
```

The -> Operator

- In C++, pointers are explicit. Given a pointer to an object, you need to dereference the pointer before selecting a field or calling a method. Given the definition of pp from the previous slide, you cannot write:

```
pp.getX();
```



because pp is not a structure or an object of a class.

- You also cannot write:

```
*pp.getX();
```



because "." takes precedence over "*". It is equivalent to:

```
*(pp.getX());
```



- To call a method given a pointer to an object, you need to write:

```
(*pp).getX();
```

```
pp->getX();
```

Please note that operator () takes precedence over '.'

Precedence

- If an expression contains more than one operator, C++ uses precedence rules to determine the order of evaluation. The arithmetic operators have the following relative precedence:

TABLE 1-4 Operators available in C++

Operators organized into precedence groups	Associativity
`()  []  ->  .`	*left*
unary operators: `-  ++  --  !  &  *  ~  (`*type*`)  sizeof`	*right*
`*  /  %`	*left*
`+  -`	*left*
`<<  >>`	*left*
`<  <=  >  >=`	*left*
`==  !=`	*left*
`&`	*left*
`^`	*left*
`|`	*left*
`&&`	*left*
`||`	*left*
`?:`	*right*
`=  op=`	*right*

The Keyword this

- In the implementation of the methods within a class, you can usually refer to the private instance variables of that class using just their names. C++ resolves such names by looking for matches in the following order (principle of proximity):
  - Parameters or local variables declared in the current method
  - Instance variables of the current object
- It is often convenient to use the same names for parameters and instance variables. If you do, you must use the keyword this (defined as a pointer to the current object) to refer to the instance variable, as in the constructor for the Point class (think of this as self in Python) :

```
Point::Point(int cx, int cy) {
    x = cx;
    y = cy;
}
```

```
Point::Point(int x, int y) {
    this->x = x;
    this->y = y;
}
```

Pointer to Function

Most modern object-oriented languages use pointers to functions internally but hide the details from the programmer. By contrast, C++ makes it possible for programmers to declare pointers to functions and then use those functions as data values in an application.




```
#include <iostream>
using namespace std;

int compare(int (*fn)(int, int), int xx, int yy) { return fn(xx, yy); }
int max(int x, int y) { return (x > y) ? x : y; }
int min(int x, int y) { return (x > y) ? y : x; }

int main()
{
  int x = 10;
  int y = 5;
  cout << "max :" << x << " & " << y << " = " << compare(max, x, y) << endl;
  cout << "min :" << x << " & " << y << " = " << compare(min, x, y) << endl;
  return 0;
}
```

Simple Arrays in C++

- We have previously used arrays in their low-level form only scarcely so far:

```
char cstr[10];
char cstr[] = "hello";
char cstr[] = { 'h', 'e', 'l', 'l', 'o', '\0' };
```

  because the Vector class is so much better.
- From the client perspective, an array is like a brain-damaged form of Vector with the following differences:
  - The only operation is selection using [];
  - Array selection does not check that the index is in range;
  - The declared length of an array is fixed at the time it is created;
  - Arrays don't store their actual length, so programs that use them must pass an extra integer value that represents the number of elements actively in use.

Simple Arrays in C++

- Array variables are declared using the following syntax:

```
type name[n];
```

  where type is the element type, name is the array name, and n is a constant integer expression indicating the length.

- Array variables can be given initial values at the time they are declared:

```
int DIGITS[] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
```

- The size of the array specified in the declaration is called the allocated size. The number of elements actively in use is called the effective size.
- To determine how many elements there are in a strange array (e.g., declared by someone else or dynamically changed):

```
sizeof MY_ARRAY / sizeof MY_ARRAY[0]
```

Pointers and Arrays

- In C++, the name of an array is synonymous with a pointer to its first element. For example, if you declare an array

```
int list[100];
```

  the C++ compiler treats the name list as a pointer to the address &list[0] whenever necessary, and list[i] is just *(list+i), because pointer arithmetic counts the objects pointed to by the pointer.

- Although an array is often treated as a pointer, they are not entirely equivalent, e.g., you can assign an array to a pointer (of the same type), but not vice versa, because an array is a non-modifiable lvalue (so are constant variables).

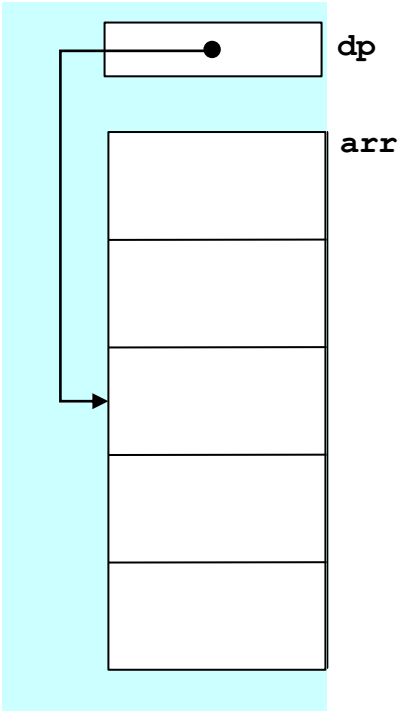
- When you pass an array to a function, only the address of the array is copied into the parameter. This strategy has the effect of sharing the elements of the array between the function and its caller (i.e., call by pointer).

Pointer Arithmetic

- Like C before it, C++ defines the + and - operators so that they work with pointers. Dangerous, though. Be careful!
- Suppose, for example, that you have made the following declarations:

```
double arr[5];
double * dp = arr;
```

How do those variables appear in memory?



- C++ defines pointer addition so that the following identity always holds (Note the following are not C++ statements!):

```
arr[i] ≡ *(arr+i) ≡ *(dp+i) ≡ dp[i]
&arr[i] ≡ arr+i ≡ dp+i ≡ &dp[i]
```

Thus, dp + 2 points to arr[2].

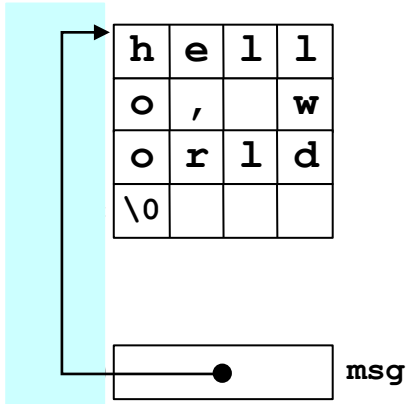
C Strings are Pointers to Characters

- As you know from Chapter 3, C++ supports the old C style of strings, which is simply a pointer to a character, which is the first element of a character array terminated by the null character ('\0').
- Given this definition, what does the declaration

```
char cstr[] = "hello, world";
char* msg = cstr;
```

generate in memory?

- You can still select characters in msg by their index because of the equivalence of arrays and pointers.

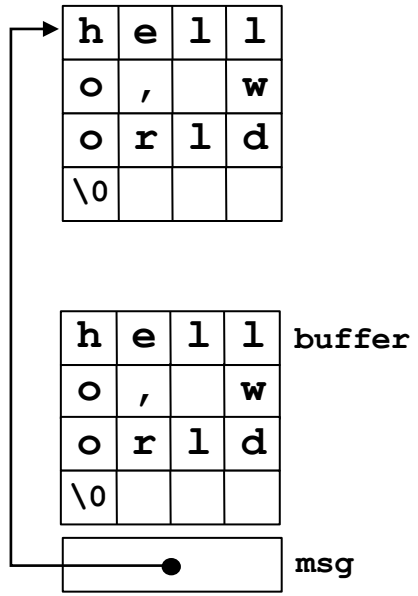


h	e	l	l	o	,	w	o	r	l	d	\0
0	1	2	3	4	5	6	7	8	9	10	11

Examples: C String Functions

1. Implement the C library function strlen(cstr) that returns the length of the C string cstr.
2. Implement the C library function strcpy(dst, src), which copies the characters from the string src into the character array indicated by dst. For example, the code on the left should generate the memory state on the right:

```
char str[] = "hello, world";
char* msg = str;
char buffer[16];
strcpy(buffer, msg);
```



C String Functions **strlen(cstr)**

```
int strlen(char str[]) {
   int n = 0;
   while (str[n] != '\0') {
      n++;
   }
   return n;
}

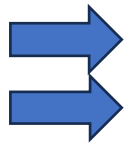
int strlen(char *str) {
   int n = 0;
   while (*str++ != '\0') {
      n++;
   }
   return n;
}

int strlen(char *str) {
   char *cp;
   for (cp = str; *cp != '\0'; cp++);
   return cp - str;
}
```

*p++

- The *p++ idiom means dereference p and return as a lvalue the object to which it currently points, and increment the value of p so that the new p points to the next element in the array.
- When you work with C++, understanding the *p++ idiom is important primarily because the same syntax comes up in STL iterators, which are used everywhere in professional code.
- It is, however, equally important that you need to be careful when using it to avoid buffer overflow errors.

```
int arr[] = {1, 2, 3, 4};
int * p = arr;
int a = *p++; // a = *p; p = p + 1;
    a = *p++;
```



```
a = 1
a = 2
```

*p++

```
#include <iostream>
#include <string>
using namespace std;

int main(void)
{
    int arr[] = {1, 2, 3, 4};
    int * p = arr;
    int a = *p++; // int a = *arr++; is this OK?
    // a = *(p++); i.e., a = *p; p = p + 1;
    int b = *++p;
    // b = *(++p); i.e., p = p + 1; b = *p;
    cout << "a = " << a << ", b = " << b << endl;
    return 0;
}

Output:
a = 1, b = 3
```

strcpy: the Hot-Shot Solution

```
void strcpy(char* dst, char* src)
{
   while (*dst++ = *src++);
}
```



```
dst[0] = src[0];
.
.
.
dst[n]=src[n] = '\0'; stop while loop
```

Note: The 'operator=' returns the left operand dst[i]

Needs to be careful when using it to avoid buffer overflow errors, when?

Pointers Example

```
int **ppi, *pi, i = 10;
pi = &i;
ppi = &pi;

i: 10                  &i: 0x6dfed4
pi: 0x6dfed4           &pi: 0x6dfed8
ppi: 0x6dfed8
*pi = i = 10

ppi: 0x6dfed8          &ppi: 0x6dfedc
*ppi: 0x6dfed4 (= pi)
**ppi: 10 (= *pi = i =10)
```

**ppi ➡ *pi ➡ i

TABLE 1-4 Operators available in C++

Operators organized into precedence groups	Associativity
`()  []  ->  .`	*left*
unary operators: `-  ++  --  !  &  *  ~  (type)  sizeof`	*right*
`*  /  %`	*left*

```
int main()
{
        int ***pppi, **ppi, *pi, i = 10;
        pi = &i;
        ppi = &pi;
        pppi = &ppi;

        cout << "pi = " << hex << pi << endl;
        cout << "&i = " << hex << &i << endl;
        cout << "ppi = " << hex << ppi << endl;
        cout << "&pi = " << hex << &pi << endl;
        cout << "pppi = " << hex << pppi << endl;
        cout << "&ppi = " << hex << &ppi << endl;

        cout << dec << " i=" << i << endl;
        cout << "*pi = " << *pi << endl;
        cout << "**ppi=" << **ppi << endl;
        cout << "***pppi=" << ***pppi << endl;
}
```

```
pi = 16fcd7354
&i = 16fcd7354
ppi = 16fcd7358
&pi = 16fcd7358
pppi = 16fcd7360
&ppi = 16fcd7360

i=10
*pi = 10
**ppi=10
***pppi=10
```

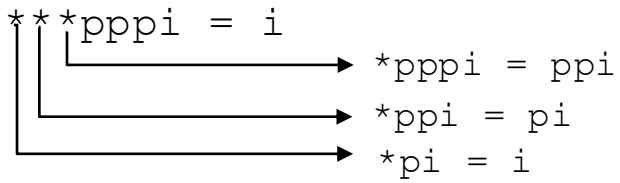


TABLE 1-4 Operators available in C++

Operators organized into precedence groups	Associativity
`()` `[]` `->` `.`	*left*
unary operators: `-` `++` `--` `!` `&` `*` `~` *(type)* `sizeof`	*right*
`*` `/` `%`	*left*
`.`	*left*